

NeoStats: AI-Powered Credit Risk Intelligence Platform

FULL-STACK ASSIGNMENT: EDA → ML MODEL → EXPLAINABLE AI →
CONVERSATIONAL INTERFACE

Presented by
Prantik sarkar
Candidate for AI engineer role

Problem Statement & Solution Matrix:

The Credit Risk Challenge: Problem Statement vs. Solution Matrix

THE UNDERWRITING FRICTION

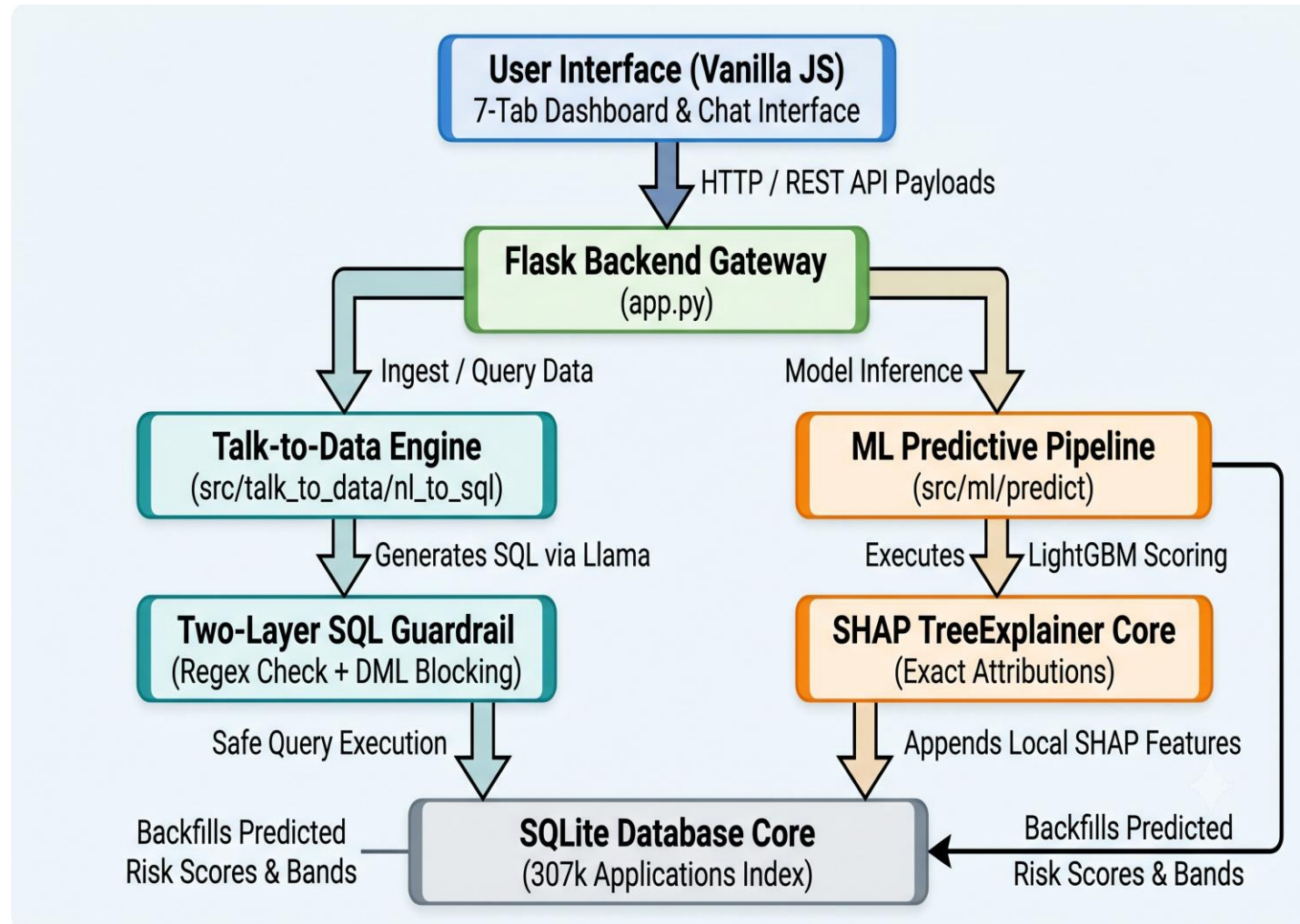
1. **Manual Latency:** Legacy underwriting relies on slow, fragmented workflows, creating massive turnaround delays and application backlogs.
2. **The "Black Box" Dilemma:** High-performance machine learning models make accurate decisions but lack auditable explanations required by financial regulators
3. **Exploration Bottlenecks:** Non-technical risk managers and business analysts depend entirely on engineering teams to write SQL and pull insight reports..
4. **Imbalance Washouts:** Standard classification strategies fail on heavily skewed datasets (~8-9% default baseline), causing catastrophic false-approvals.

THE NEOSTATS ORCHESTRATION

1. **Automated Pipeline:** Houses a fully containerized pipeline that automates raw CSV data ingestion, feature scaling, and inference in seconds.
2. **Exact Explainability Core:** Integrates a deterministic SHAP TreeExplainer that translates statistical math weights into clear business-read text.
3. **Conversational Data Agent:** Implements a sandboxed, guard-railed Natural Language-to-SQL engine via OpenRouter, democratizing instant data access.
4. **Cost-Sensitive Analytics:** Instantiates cost-penalized loss metrics and tunes decision thresholds explicitly to capture high-risk minority defaults.

System Architecture:

The overall architecture of the system as follows



- **Decoupled Dual-Track Pipeline:** Separates real-time credit scoring (LightGBM) from analytical querying (NL→SQL), preventing resource contention and optimizing throughput.
- **Continuous Compliance Audit:** Automatically binds model inferences to a deterministic SHAP TreeExplainer, instantly backfilling risk metrics and local feature attributions directly into SQLite.
- **Sandboxed Query Security:** Enforces multi-layer database protection by combining LLM schema-grounding with rigid regex filters and absolute Data Modification Language (DML) blocking.

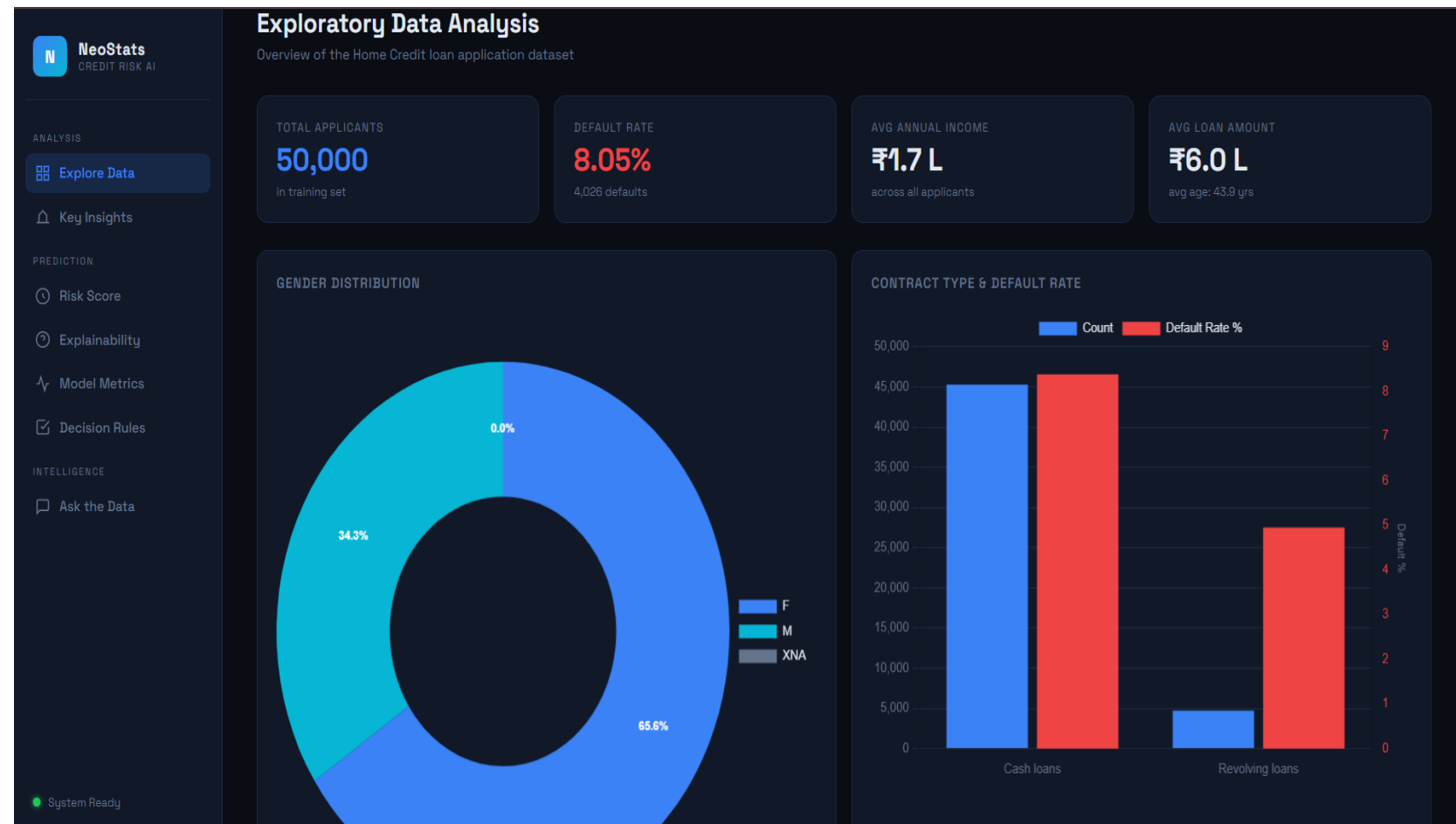
Exploratory Data Analysis:

Population Scale & Skew: Analyzed 307,511 application records, identifying an **8.1% baseline default rate** that dictates our cost-sensitive modeling choices.

Informative Noise Signals:

Discovered that missing data structures within external credit scoring blocks are non-random and act as predictive proxies for higher risk.

Structural Exposure Sinks: Mapped clear risk boundaries showing that debt allocations surpassing **3× total annual earnings** yield exponential default frequencies.



Key Insights Console:

Empirical Risk Drivers & Feature Interactions



Data reveals default rates double for under-30 applicants, spike above 20% when external credit scores drop below 0.35, and amplify heavily in low-skill occupational sectors.



Machine Learning Layer:

Cost-Sensitive LightGBM & Cross-Validation Framework

Imbalance & Optimization Architecture: Features a cost-sensitive LightGBM engine with balanced class weights, tuned via an out-of-fold grid search to maximize the F1-score and isolate high-risk defaults.

Empirical Validation Benchmarks: Achieved an **ROC-AUC of 0.7447** and a minority-optimized **PR-AUC of 0.2344** across a rigid, 5-fold stratified cross-validation framework.

Calibrated Operational Cutoff: Rejected the default 0.5 split in favor of a mathematically optimized **0.61 classification threshold** to safeguard the platform's portfolio underwriting margins.

Real-Time Risk Scoring:

Automated Real-Time Underwriting Console

- Real-Time Predictive Payload:** Forms dynamically serialize applicant parameters directly into the pipeline, outputting a precise default probability, a scaled score (\$0-100\$), and a categorical risk band (Low, Medium, High).

- Deterministic Underwriting Policy:** Transitions credit decisions from manual review to automated pipelines by instantly enforcing the mathematically optimized **61% rejection threshold**.

- Database Ledger Sync:** Automatically persists every generated risk calculation and prediction history array back into the SQLite relational database core for downstream portfolio logging.

The screenshot displays the NeoStats Credit Risk AI console interface. It features a sidebar with navigation options: ANALYSIS (Explore Data, Key Insights), PREDICTION (Risk Score, Explainability, Model Metrics, Decision Rules), and INTELLIGENCE (Ask the Data). The main area is divided into three sections: APPLICANT INFORMATION, RISK ASSESSMENT RESULT, and TOP RISK DRIVERS (SHAP).

APPLICANT INFORMATION:

ANNUAL INCOME (₹)	LOAN AMOUNT (₹)
270000	500000

ANNUITY AMOUNT (₹)	GOODS PRICE (₹)
25000	450000

AGE (YEARS)	YEARS EMPLOYED
35	5

GENDER	CONTRACT TYPE
Male	Cash loans

EDUCATION	INCOME TYPE
Secondary / secondary special	Working

OWN CAR	OWN REALTY
Yes	Yes

EXT. SCORE 1 (0-1)	EXT. SCORE 2 (0-1)
0.5	0.5

EXT. SCORE 3 (0-1)	FAMILY MEMBERS
0.5	2

RISK ASSESSMENT RESULT:

A gauge shows a score of 29.4, categorized as Low Risk. Below the gauge, the following metrics are displayed:

PROBABILITY	RISK BAND
29.4%	Low

THRESHOLD	DECISION
61%	APPROVE

A green button indicates **✓ LOAN APPROVED**.

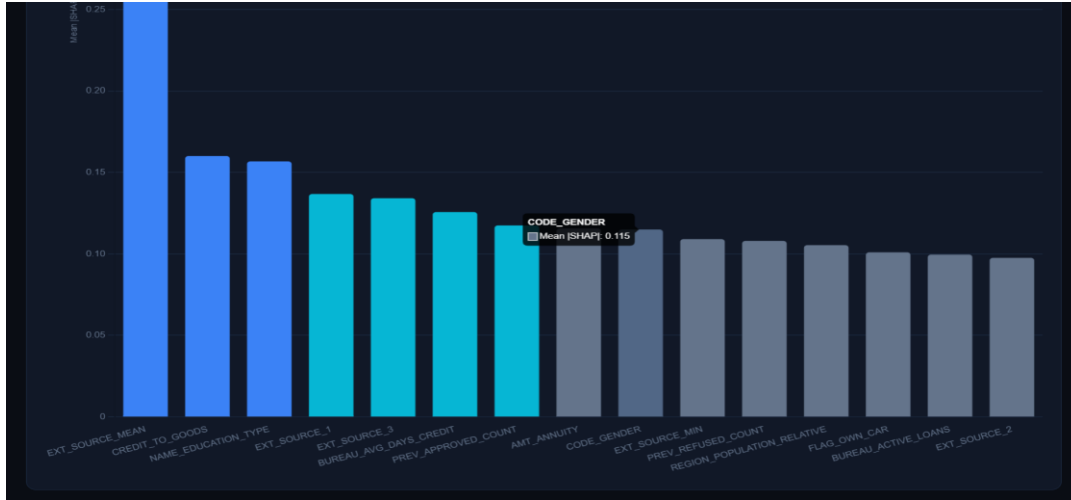
TOP RISK DRIVERS (SHAP):

Feature	Value
PREV_AVG_AMT_APPLICAT_...	-0.533
BUREAU_TOTAL_CREDIT_S...	+0.374
FLAG_OWN_CAR	-0.286
PREV_APPROVED_COUNT	+0.196
EXT_SOURCE_MIN	-0.178

The bottom status bar indicates "System Ready".

Explainable AI (XAI):

Exact Local Feature Attributions via SHAP TreeExplainer



- Mathematical Attribution Rigor:** Leverages a deterministic **SHAP TreeExplainer** core to extract exact, additive local feature weights directly from tree ensemble branches, bypassing unstable kernel approximations.

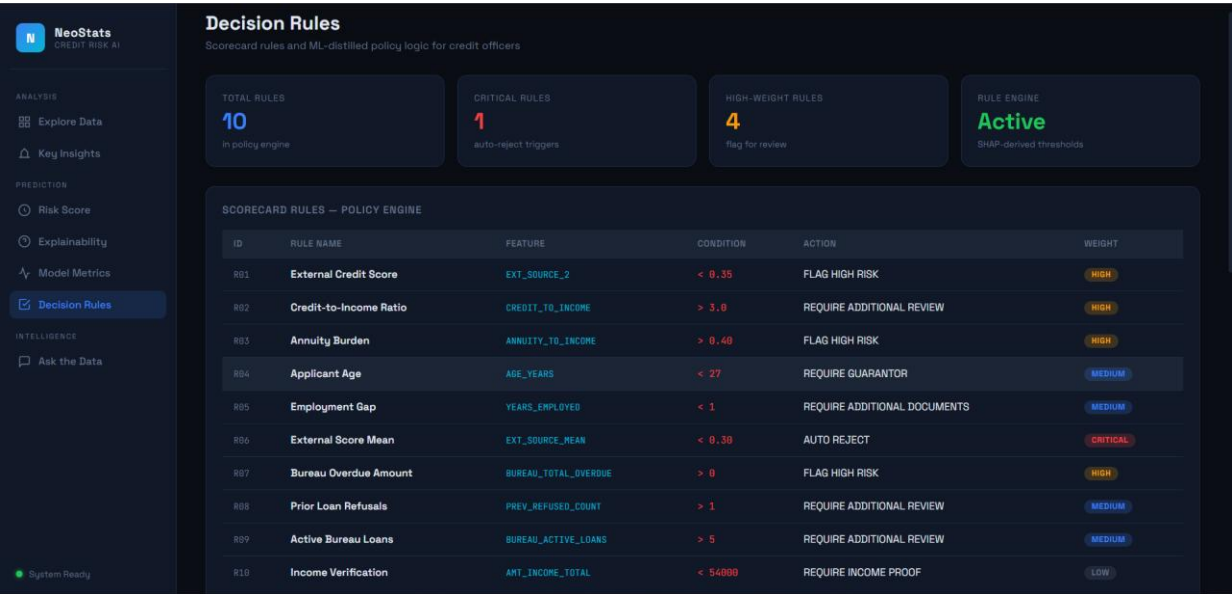
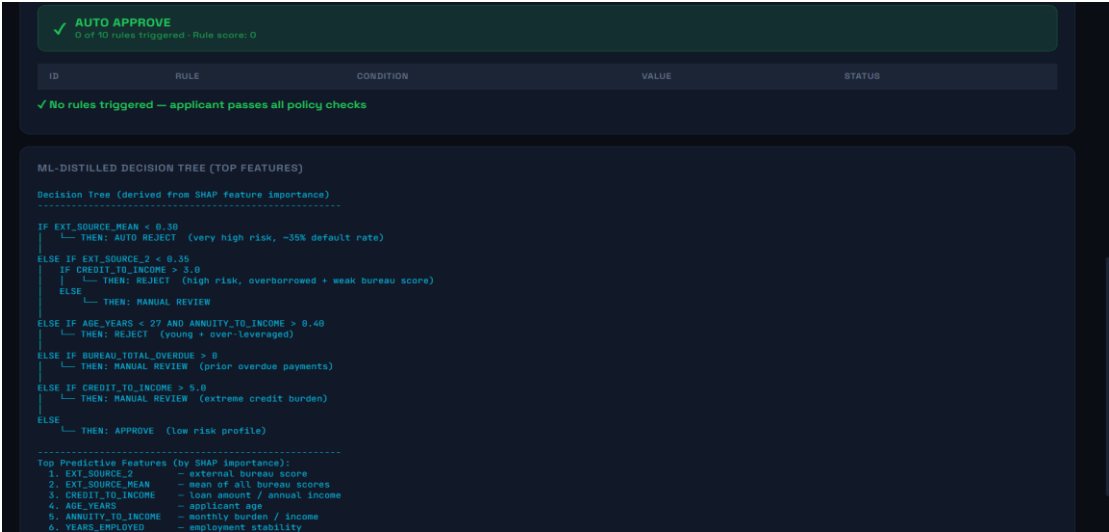
- Semantic Risk Direction Mapping:** Automatically translates highly complex statistical weights into intuitive, business-readable visual indicators: red bars denote risk amplification, while green bars map risk mitigation.

- Transparent Audit Pathways:** Demystifies individual underwriting outputs by letting compliance teams trace decisions instantly down to primary continuous hooks, such as CREDIT_TO_INCOME.

Decision Rules:

Grounding Insights into Executable Compliance Policy

- Deterministic Policy Overrides:** Bridges the gap between statistical machine learning models and absolute institutional guardrails by enforcing hardcoded, auditable risk boundaries.
- Critical Exposure Redlines:** Automatically enforces an absolute **AUTO REJECT** override the moment an applicant's computed EXT_SOURCE_MEAN drops below **0.30**, completely bypassing standard inference pathways.
- Tiered Escalation Workflows:** Flags applications for manual human underwriting review if requested debt balances exceed **3× annual income**, or enforces a guarantor requirement if the applicant is under **27 years old**.



The Conversational "Ask the Data" Interface:

Conversational Analytics & Business Intelligence Integration

The screenshot displays the 'NeoStats CREDIT RISK AI' application. The main heading is 'Ask the Data', with a subtitle 'Natural-language interface — powered by LLM + SQL'. Below this, a 'TRY ASKING:' section contains several buttons: 'What is the overall default rate?', 'Default rate by gender', 'Highest default by income', 'Top 5 occupations', 'High-risk applicants', and 'Loan by education'. The selected button, 'Loan by education', has triggered a response. The response includes a natural language summary: 'The average loan amount varies significantly by education level. Professionals with higher education tend to have higher average loan amounts, while those with lower education levels have lower averages. This suggests education level may influence borrowing behavior.' Below the summary is a SQL query block:

```
SELECT
  NAME_INCOME_TYPE,
  AVG(ART_GOODS_PRICE) AS AVG_AMOUNT
FROM
  applications
GROUP BY
  NAME_INCOME_TYPE
ORDER BY
  AVG_AMOUNT DESC;
```

 Underneath the SQL is a table with two columns: 'NAME_INCOME_TYPE' and 'AVG_AMOUNT'. The table contains five rows of data. At the bottom, there is a text input field with the placeholder 'e.g. What percentage of applicants own a car?' and a blue send button. The left sidebar shows navigation options: 'ANALYSIS' (Explore Data, Key Insights), 'PREDICTION' (Risk Score, Explainability, Model Metrics, Decision Rules), and 'INTELLIGENCE' (Ask the Data). The bottom status bar indicates 'System Ready'.

NAME_INCOME_TYPE	AVG_AMOUNT
Businessman	1800000
Maternity leave	785000
Unemployed	648000
State servant	613326.3064102564
Commercial associate	601488.9891652942

- Democratization of Data Access:**

Translates conversational plain English inputs directly into highly optimized database commands, giving non-technical risk analysts instant access to database analytics.

- Schema-Grounded Query Translation:**

Seamlessly pipes natural queries through OpenRouter using lightweight instruction models (llm-2.5-1.2b) to build strict, schema-matched SQLite query blocks.

- Dual Presentation Delivery:**

Processes structural database results out-of-thread to simultaneously render organized, filterable UI data tables alongside natural language analytical summaries.

Talk-to-Data Security & Hallucination Defense:

Secure Translation Architecture & LLM Guardrails

Core Defensive Subsystems:

- Contextual Schema Grounding:** Injects explicit, relational database blueprints directly into the system prompt. This completely eliminates arbitrary model hallucinations and anchors the LLM strictly to real tables and columns.
- Rigid Output Constraints:** Enforces a non-negotiable instruction format demanding raw code string returns (*"Return ONLY a valid SQLite SQL query"*). This fundamentally blocks conversational prose leakage or markdown text contamination.
- Deterministic Input & Output Filtering:** Runs an instantaneous, two-layer defensive validation check. Every query must clear an automated regex syntax scan and a strict Data Modification Language (DML) blocker that drops any command containing unauthorized keywords (`INSERT`, `UPDATE`, `DELETE`, `DROP`).
- Hardcoded Performance Boundaries:** Protects the relational storage engine from resource exhaustion or runaway table scans by automatically appending a hardcoded `LIMIT 500` constraint to all translated execution targets.

Core Technology & Design Matrix:

Engineering Inventory & Selection Rationales

Layer Stack	Technology Selected	Technical Selection Rationale
Predictive Engine	LightGBM	Superior convergence speeds on imbalanced tabular matrix frames; native categorical optimization.
Explainable AI	SHAP TreeExplainer	Yields exact, rapid additive local importance values tailored specifically for tree ensembles.
LLM Gateway	OpenRouter Framework	Eliminates up-front credential overhead via a clean API; utilizes liquid/lfm-2.5-1.2b-instruct.
Database Core	SQLite Instance	Embedded database footprint; eliminates configuration friction for local evaluation execution.
API Backend	Flask Framework	High-velocity, lightweight REST gateway offering minimal resource overhead.
UI Presentation	Vanilla JS / Chart.js	Eliminated heavy JS framework compilations (React/Vue) for clean, low-latency dashboard performance.
Infrastructure	Docker / Compose	Guarantees environmental immutability across evaluation environments in a single command.

Technical Design Decisions

Architectural Trade-Offs & Engineering Intuition

- **Ensemble Strategy (LightGBM over XGBoost):** Selected LightGBM due to its significantly faster training throughput, lower memory consumption, and native optimization for high-cardinality splits on distributed, un-imputed tabular financial data.
- **LLM Orchestration (OpenRouter + Llama):** Chose OpenRouter to abstract model routing under a unified interface. Employed a lightweight instruction model (llm-2.5-1.2b-instruct) to achieve lightning-fast text-to-SQL conversions with zero upfront runtime costs or credit card requirements.
- **Storage Footprint (SQLite over PostgreSQL):** Decoupled and isolated database configurations to leverage an embedded SQLite database engine. This minimizes setup friction for local evaluation execution, allowing the reviewer to skip spinning up heavy database servers.
- **Boundary Calibration (Threshold 0.61 over 0.5):** Rejected standard, balanced probability margins in favor of a customized threshold calibrated through out-of-fold cross-validation grid searches to maximize portfolio F_1 -scores.
- **Deterministic Explainability (TreeExplainer over KernelExplainer):** Exploited the model's underlying structural tree architecture to deploy SHAP TreeExplainer, bypassing high-variance kernel approximations to compute mathematically exact local feature weights.

Future Scope & Long-Term Roadmap:

Production Scaling & Advanced System Evolution

- Enterprise Storage Migration:** Swap the embedded database core for an enterprise **PostgreSQL** or distributed cloud database instance to unlock high-concurrency analytical workloads and parallel query processing.
- Asynchronous Task Architecture:** Integrate a **Celery** infrastructure worker layer backed by a **Redis** caching tier to offload and stream resource-heavy model retraining operations completely out-of-thread.
- ML Lifecycle & Lineage Tracking:** Embed an **MLflow** orchestration framework to automatically log model hyperparameter variants, preserve artifact lineage maps, and actively monitor prediction drift over time.
- Multi-Framework Attributions:** Deploy **LIME** (Local Model-Agnostic Explanations) alongside our existing SHAP engine to provide risk compliance teams with comparative, multi-angle diagnostic insights.
- Unified Access Controls:** Wrap the Flask API backend in a robust **JWT (JSON Web Token)** or **OAuth2** authentication gateway to secure exposed endpoints from unauthorized local or network clients.

Repository Blueprint & Architecture Deployment

Technical Deliverables & Source Configuration

- Open-Source Codebase:** github.com/Prantik400/credit_risk_platform
- Development Stack Ecosystem:** Built natively using Python, LightGBM, SHAP, OpenRouter (Llama), Flask, and SQLite.
- Environmental Packaging:** Containerized completely via **Docker** and **Docker Compose** to guarantee environment immutability.
- Single-Command Initialization:** Spin up the entire multi-container architecture locally with zero manual dependency configuration:
Bash
`docker-compose up --build`

Thank You !